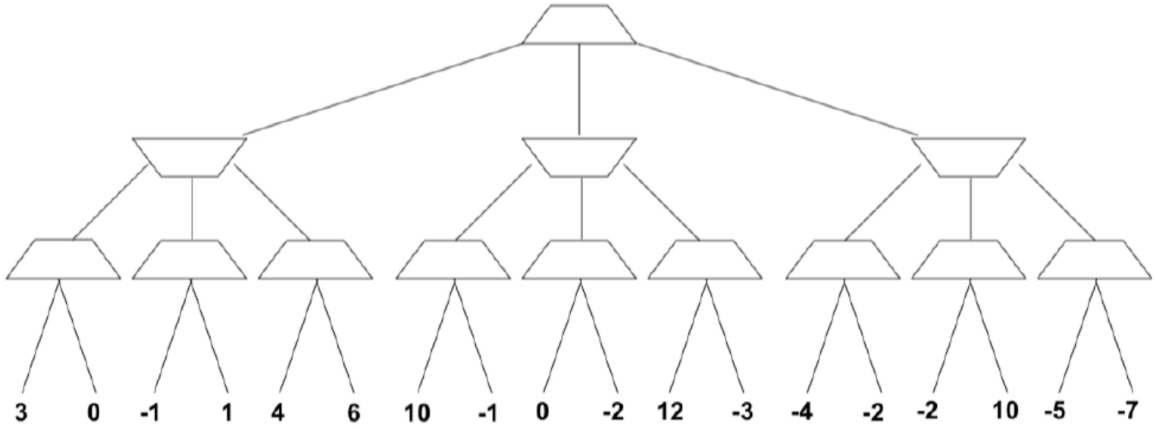
BÀI TẬP LT: TÌM KIẾM, CSP VÀ GAME TRONG AI

Họ và tên: Đỗ Duy Quý
MSSV: 3123560073

Bài 1. **Minimax và Alpha-Beta Pruning**

Xét cây trò chơi ở Hình 1: gốc là MAX, tầng thứ nhất là MIN và tầng kế tiếp là MAX. Các con được duyệt theo thứ tự từ trái sang phải. Ta sử dụng cùng cấu trúc cây để giải bằng Minimax và kiểm tra hiệu quả cắt tỉa Alpha-Beta.



Hình 1. Cây Minimax và Alpha-Beta (trích từ đề BT03).

Các cặp giá trị lá theo thứ tự trên hình là (3, 0), (-1, 1), (4, 6) ở nhánh trái; (10, -1), (0, -2), (12, -3) ở nhánh giữa; và (-4, -2), (-2, 10), (-5, -7) ở nhánh phải.

a) Tính các giá trị Minimax

Trước hết, xét các nút MAX ngay phía trên tầng lá và lấy giá trị lớn hơn trong từng cặp:

Nhóm trái: max(3, 0) = 3; max(-1, 1) = 1; max(4, 6) = 6.

Nhóm giữa: max(10, -1) = 10; max(0, -2) = 0; max(12, -3) = 12.

Nhóm phải: max(-4, -2) = -2; max(-2, 10) = 10; max(-5, -7) = -5.

Tiếp theo, tại tầng MIN: nhánh trái nhận min(3, 1, 6) = 1; nhánh giữa nhận min(10, 0, 12) = 0; nhánh phải nhận min(-2, 10, -5) = -5.

Cuối cùng, nút gốc MAX nhận max(1, 0, -5) = 1.

b) Lựa chọn tối ưu của MAX

Vì 1 là giá trị lớn nhất trong ba nhánh, MAX sẽ chọn nhánh bên trái. Giá trị Minimax của trò chơi là 1.

c) Alpha-Beta Pruning khi duyệt trái sang phải

Tại gốc, khởi tạo $\alpha = -\infty$ và $\beta = +\infty$. Mỗi khi xét nút MAX, ta cập nhật cận dưới α; ở nút MIN, ta cập nhật cận trên β. Nếu $\alpha \geq \beta$, những con chưa duyệt có thể bỏ qua vì không còn ảnh hưởng đến lựa chọn ở gốc.

Bước 1 - Nhánh MIN trái: Hai nút MAX đầu lần lượt trả về 3 và 1, nên β giảm từ $+\infty$ xuống 3 rồi xuống 1. Ở nút MAX thứ ba, lá đầu có giá trị 4; do $4 \geq \beta = 1$, ta cắt lá còn lại có giá trị 6. Kết quả của nhánh MIN trái là 1.

Bước 2 - Nhánh MIN giữa: Gốc đã có $\alpha = 1$. Nút MAX đầu trả 10 nên $\beta = 10$; nút MAX tiếp theo trả 0 nên $\beta = 0$. Vì $\beta = 0 \leq \alpha = 1$, toàn bộ nút MAX cuối (gồm hai lá 12 và -3) bị cắt.

Bước 3 - Nhánh MIN phải: Nhánh này cũng nhận $\alpha = 1$. Nút MAX đầu trả -2 nên $\beta = -2$. Vì $\beta = -2 \leq \alpha = 1$, có thể bỏ qua hai nút MAX còn lại và các lá (-2, 10), (-5, -7).

Ở những nhánh đã bị cắt, giá trị ghi nhận trong lúc duyệt có thể chỉ là một cận đủ để quyết định, chưa hẳn là giá trị Minimax chính xác nếu mở rộng toàn bộ cây.

d) Số nút lá được đánh giá

Các lá không cần thăm là: lá 6 ở nhánh trái; hai lá 12 và -3 ở nhánh giữa; bốn lá -2, 10, -5, -7 ở nhánh phải. Tổng cộng có 7 lá được cắt.

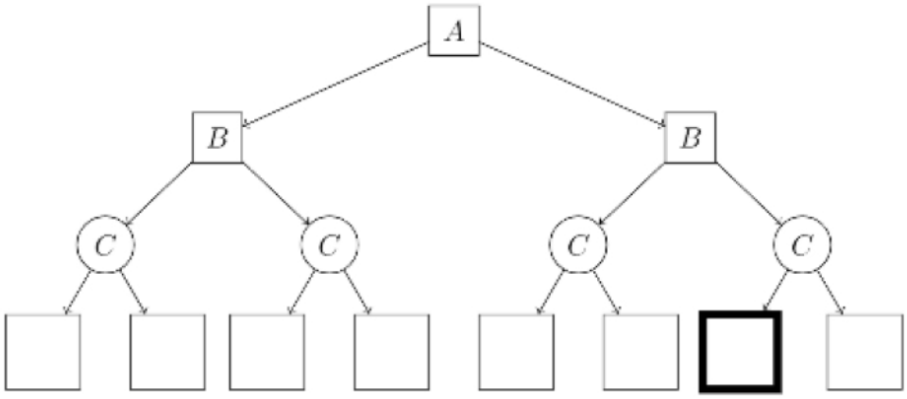
Minimax duyệt toàn bộ 18 nút lá, còn Alpha-Beta chỉ cần đánh giá 11 lá. Như vậy số lá phải duyệt giảm 7/18, tương đương khoảng 38,9%, trong khi quyết định tối ưu tại gốc vẫn không đổi.

Bài 2. **Cắt tỉa trong cây có nút ngẫu nhiên (Expinimax)**

Hình 2 gồm ba cây Expinimax. Trong cả ba trường hợp, A là nút MIN, B là nút MAX, C là nút ngẫu nhiên với hai nhánh có xác suất bằng nhau. Ta duyệt từ trái sang phải và xét khả năng bỏ qua nút lá viền đậm; nếu hai lựa chọn ngang giá trị thì không cắt.

Để chứng minh, tôi dùng các bộ giá trị lá minh họa. Các con số này được chọn thêm để kiểm tra điều kiện cắt tỉa, không phải giá trị được cho sẵn trong đề. Ký hiệu "?" đại diện cho lá chưa cần xét.

a) Trường hợp 1: lá đầu tiên của C thứ hai, dưới B bên phải

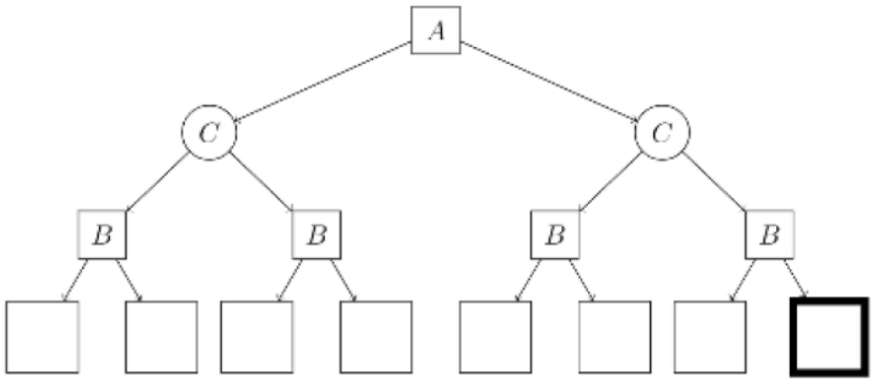


Hình 2a. Trường hợp cắt tỉa thứ nhất.

Trường hợp này có thể cắt. Chẳng hạn, bốn lá dưới nút B trái đều bằng 0 nên B trái có giá trị 0. Tại B phải, nút C thứ nhất có hai lá cùng bằng 2 và có kỳ vọng 2. Vì B phải là MAX, giá trị của nó ít nhất đã bằng 2. Nút A là MIN, đã có phương án bên trái bằng 0 nên chắc chắn không chọn nhánh phải có giá trị lớn hơn 0. Ta có thể bỏ qua toàn bộ C thứ hai của B phải, bao gồm lá viền đậm.

Ví dụ gán: B trái = [0, 0 | 0, 0]; B phải = [2, 2 | ?, ?]. Điều kiện cắt ở đây là cận dưới của B phải lớn hơn giá trị B trái.

b) Trường hợp 2: lá cuối của B thứ hai dưới C bên phải



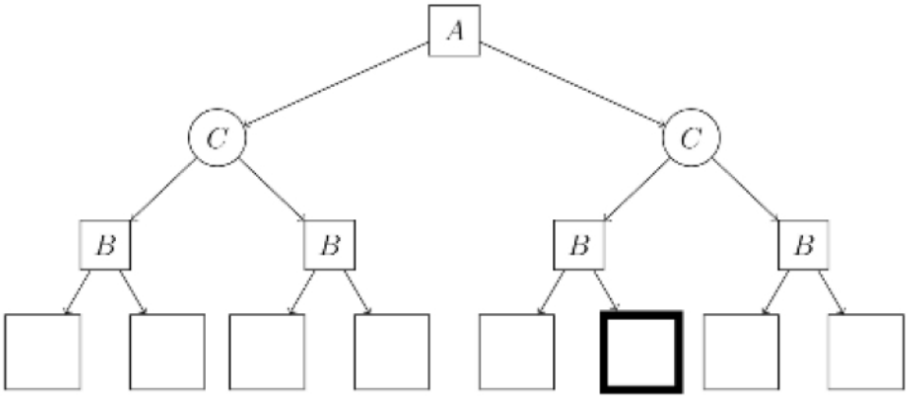
Hình 2b. Trường hợp cắt tỉa thứ hai.

Trường hợp 2 cũng cắt được mà không cần biết trước giới hạn toàn cục của giá trị lá. Giả sử bốn lá dưới C trái đều bằng 0 nên C trái trả 0. Dưới C phải, nút B thứ nhất có hai lá bằng 2 nên trả 2; nút B thứ hai đã xét lá đầu bằng 2, do đó giá trị MAX của nút này không thể nhỏ hơn 2, dù chưa biết lá cuối.

Ví dụ: C trái [0, 0 | 0, 0]; C phải [2, 2 | 2, ?].

Do hai con của C phải có xác suất 1/2, kỳ vọng của C phải ít nhất là (2 + 2)/2 = 2. Vì A là MIN và đã có nhánh C trái bằng 0, nhánh C phải không thể thắng. Lá cuối cùng có viền đậm vì thế không cần duyệt.

c) Trường hợp 3: lá thứ hai của B thứ nhất dưới C bên phải



Hình 2c. Trường hợp cắt tỉa thứ ba.

Trường hợp 3 không thể bảo đảm cắt lá viền đậm khi đề chưa cho giới hạn giá trị lá. Sau khi xét lá đầu của B thứ nhất dưới C phải, ta chỉ có cận dưới cho B này. Nút B còn lại của C phải chưa được duyệt và có thể nhận giá trị âm tùy ý nhỏ. Vì vậy kỳ vọng của C phải vẫn có thể giảm xuống dưới kết quả nhánh C trái; việc cắt lá ở thời điểm này có nguy cơ thay đổi quyết định tại A.

Nếu có thêm một khoảng giá trị hữu hạn cho mọi lá, ta mới có thể tính cận trên và cận dưới của kỳ vọng để xem có đủ điều kiện cắt hay không. Đó là giả định bổ sung, không có trong đề gốc.

d) So sánh với Alpha-Beta thông thường

Ở Minimax thuần túy, các nút chỉ thực hiện min hoặc max, nên một cận tốt hoặc xấu đã biết có thể đủ loại bỏ các nhánh còn lại. Với nút chance, giá trị được tính bằng trung bình có trọng số $V(C) = \sum p_i V(i)$; phần chưa duyệt vẫn có thể làm thay đổi kỳ vọng. Vì thế, cắt tỉa trong Expinimax đòi hỏi lập luận về cận của cả cây con, như hai ví dụ ở câu a và b, chứ không thể máy móc áp dụng quy tắc Alpha-Beta thông thường.

Bài 3. **Các nhận định đúng/sai về Minimax và Expectimax**

Ký hiệu V_M là giá trị Minimax của cây có gốc MAX. Khi thay mỗi nút MIN bằng một nút ngẫu nhiên với xác suất đã biết và giữ nguyên các lá, giá trị tương ứng của cây mới là V_E.

a) Luôn có $V_M \le V_E$.

Đúng. Ở mỗi nút, giá trị nhỏ nhất không lớn hơn giá trị trung bình có trọng số: $\min(x_1, \ldots, x_k) \le \sum p_i x_i$. Kết hợp quy tắc MAX giữ bất đẳng thức này, suy ra $V_M \le V_E$ (khi cấu trúc cây và giá trị lá được giữ nguyên).

b) Chính sách Minimax tối ưu bảo đảm lợi ích kỳ vọng ít nhất V_M.

Đúng. Chính sách Minimax được chọn để bảo đảm mức lợi ích trường hợp xấu nhất bằng V_M. Đối thủ/ngẫu nhiên chỉ chọn một phân phối trên những hành động hợp lệ, nên kỳ vọng khi theo chính sách này không thấp hơn mức bảo đảm đó.

c) Chính sách Minimax tối ưu bảo đảm lợi ích kỳ vọng ít nhất V_E.

Sai. Chính sách Minimax có thể khác chính sách tối đa hóa kỳ vọng của Expectimax. Chẳng hạn, ở gốc MAX: hành động 1 cho lợi ích chắc chắn 2; hành động 2 cho 0 hoặc 10 với xác suất 1/2. Minimax chọn hành động 1, kỳ vọng 2, trong khi Expectimax chọn hành động 2 với kỳ vọng 5 = V_E. Vì 2 < 5 nên mệnh đề sai.

d) Trò chơi một người có kết quả hành động ngẫu nhiên; thắng +1, thua -1

Các phương án phù hợp: Expectimax với tiện ích cuối +1/-1; và Value Iteration với phần thưởng trung gian 0, kết quả thắng/thua lần lượt +1/-1.

Expectimax dùng được khi cây trạng thái hữu hạn hoặc có một độ sâu kết thúc xác định và biết xác suất ở các nút chance. Value Iteration dùng cho mô hình MDP với trạng thái, hành động, chuyển trạng thái và phần thưởng được xác định; nếu có chu trình thì phải đặt điều kiện hội tụ phù hợp (ví dụ hệ số chiết khấu < 1 hoặc MDP kết thúc đúng nghĩa). Minimax không phải mô hình tự nhiên cho một môi trường thuần ngẫu nhiên, vì không có đối thủ chủ động cố giảm lợi ích.

Bài 4. **CSP - Đồ thị ràng buộc và lan truyền miền**

Xét bài toán xếp chỗ cho sáu sinh viên A, B, C, D, E, F. Các cặp có ràng buộc nhị phân là (A,B), (A,C), (A,D), (B,C), (B,E) và (C,F). Từ đó có thể xác định quan hệ láng giềng của từng biến.

a) Biểu diễn đồ thị ràng buộc

Tập đỉnh: {A, B, C, D, E, F}. Tập cạnh: {AB, AC, AD, BC, BE, CF}. Có thể đọc như danh sách kề:

A: B, C, D; B: A, C, E; C: A, B, F; D: A; E: B; F: C.

b) Gán A rồi dùng Forward Checking

Các miền có thể thay đổi: B, C, D. Forward Checking chỉ xét những biến chưa gán có liên kết trực tiếp với A, nên chưa truyền tác động từ B sang E hay từ C sang F.

c) Sau A, tiếp tục gán B và dùng Forward Checking

Các miền có thể thay đổi ở lần này: C và E. A đã có giá trị nên không còn là biến chưa gán; C, E là hai láng giềng chưa gán của B.

d) Gán A và duy trì Arc Consistency (MAC)

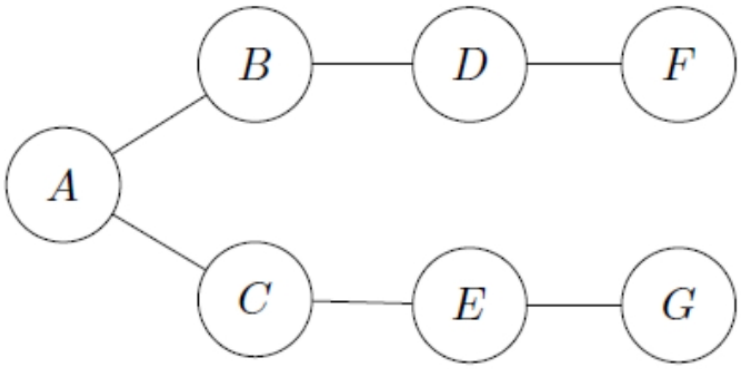
Miền của B, C, D có thể bị giảm trực tiếp. Sau đó, các thay đổi ở B và C có thể lan tiếp sang E và F. Như vậy tập các biến có khả năng bị tác động là {B, C, D, E, F}; đây là khả năng thay đổi chứ không có nghĩa mọi miền chắc chắn giảm, vì đề chưa cho quan hệ ràng buộc cụ thể.

e) Sau A và MAC, gán B rồi tiếp tục MAC

B có thể làm thay đổi miền của C và E; từ C, ảnh hưởng có thể tiếp tục sang F. A đã gán và D chỉ nối với A nên D không bị thu hẹp thêm do B trong bước này, nếu các ràng buộc nhị phân không đổi. Tập có khả năng thay đổi tiếp là {C, E, F}.

f), g), h) Phân tích backtrack với sơ đồ dạng cây ở Hình 3

Từ câu f đến h, đề dùng một đồ thị khác gồm bảy biến (Hình 3), với các cạnh AB, BD, DF, AC, CE, EG. Mỗi biến ban đầu có d giá trị. Thuật toán thực hiện Arc Consistency lúc khởi đầu và sau các lần gán thứ 2, 4, 6. Ta giả sử nhất quán cung được duy trì trên toàn bộ những cung còn liên quan để phân tích cận số lần quay lui.



Hình 3. Đồ thị ràng buộc dạng cây sử dụng ở câu f, g, h (trích từ đề BT03).

Với đồ thị có dạng cây, khi các miền đều khác rỗng và mọi cung đã nhất quán, mỗi giá trị còn lại của một biến đều có khả năng mở rộng thành một nghiệm của cây nếu xét theo thứ tự thích hợp. Tính chất này giúp hạn chế phải quay lui sâu sau mỗi lần chạy Arc Consistency.

f) A → B → C → D → E → F → G: cặp đầu (A,B) nối trực tiếp, có thể cần thử đến d giá trị ở B. Sau khi AC trên A,B, các nhánh chưa gán tách thành nhánh C-E-G và D-F. Cặp (C,D) và (E,F) ở hai nhánh tách biệt nên các giá trị còn lại phối hợp được. Gán G cuối cùng từ miền đã nhất quán. Số lần thử sai do backtrack không quá d - 1 trong cách đếm mỗi lần loại một giá trị của biến thứ hai; bậc O(d).

g) G → E → C → A → B → D → F: các cặp (G,E), (C,A), (B,D) lần lượt là hai biến có cạnh trực tiếp; ở mỗi cặp, giá trị của biến thứ hai có thể phải thử tối đa d lần trước khi AC thành công. Sau cùng gán F. Cận trên số lần thử sai là 3(d - 1), thuộc O(d).

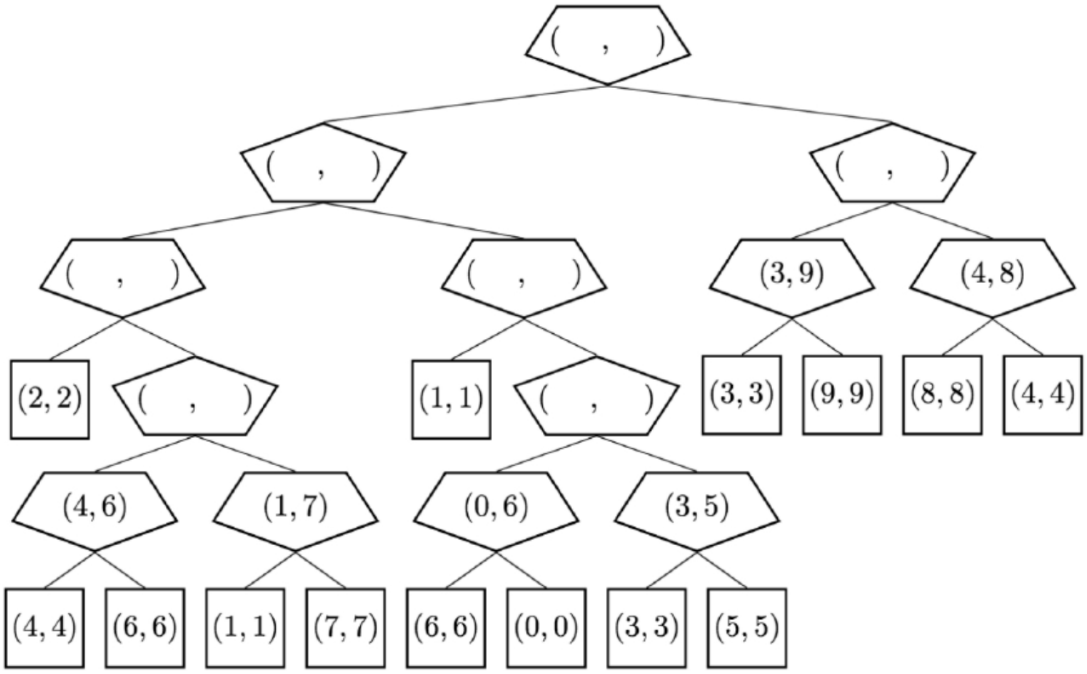
h) G → F → E → D → C → B → A: các cặp (G,F), (E,D), (C,B) không có cạnh trực tiếp, nhưng vẫn có thể bất tương thích gián tiếp thông qua phần cây chưa gán. AC sau mỗi cặp sẽ phát hiện mâu thuẫn. Do tính chất cây và AC đầy đủ, biến thứ nhất vẫn luôn có thể được mở rộng, nên mỗi cặp cần thử tối đa d lựa chọn ở biến thứ hai. Cận trên vẫn là 3(d - 1), tức O(d), dù khả năng thử sai có thể cao hơn f).

Cần phân biệt số lần backtrack và chi phí xử lý AC: mỗi lần AC có thể phải kiểm tra nhiều cặp giá trị trong các miền (phụ thuộc thuật toán, thường có nhân tử d^2 trở lên). Đề không cho quan

hệ nhị phân cụ thể nên không tính được chính xác số backtrack; các cận trên vừa nêu dựa vào giả định duy trì AC toàn cục trên cả những biến đã gán, không chỉ Forward Checking.

Bài 5. **Pacman và Casper**

Trong trò chơi ở Hình 4, Pacman luôn muốn tối đa hóa tiện ích nhưng lại tưởng Casper đang cố giảm điểm của mình. Thực tế Casper là đồng minh và cũng muốn làm tăng tiện ích cho Pacman. Mỗi nút mang cặp (u, v): u là giá trị theo cách Pacman dự đoán, còn v phản ánh kết quả hành vi thực tế của Casper. Tại các lá, hai giá trị trùng nhau.



Hình 4. Cây trò chơi Pacman - Casper (trích từ đề BT03).

a) Điền các cặp (u, v) còn thiếu

Các nút Casper dưới cùng (trái sang phải) nhận những cặp lá (4,4)/(6,6); (1,1)/(7,7); (6,6)/(0,0); (3,3)/(5,5). Chúng lần lượt có kết quả (4,6), (1,7), (0,6), (3,5): u lấy min vì Pacman nghĩ Casper đối kháng, v lấy max vì Casper thực tế thân thiện.

Hai nút Pacman ở tầng kế tiếp trong nhánh trái lần lượt chọn các cặp:

Nút thứ nhất của nhánh trái: max theo u giữa (4,6) và (1,7) là (4,6); so với lá (2,2), tiếp tục lấy (4,6).

Nút thứ hai của nhánh trái: max theo u giữa (0,6) và (3,5) là (3,5); so với lá (1,1), tiếp tục lấy (3,5).

Hai nút Pacman bên phải cây nhận sẵn các cặp (3,9) và (4,8) như trên Hình 4.

Tại Casper tầng trên bên trái: u = min(4,3) = 3, còn v = max(6,5) = 6, nên điền (3,6). Tại Casper tầng trên bên phải: u = min(3,4) = 3 và v = max(9,8) = 9, nên điền (3,9).

Tại gốc, Pacman so sánh giá trị u = 3 ở cả hai nhánh. Nếu phá hòa theo thứ tự trái trước, ta đi sang trái và ghi (3,6); nếu chọn phải thì được (3,9). Vì đề không nêu quy tắc phá hòa, cần trình bày cả hai khả năng thay vì gán duy nhất một giá trị v cho gốc.

b) Có thể cắt tỉa nhánh nào khi xét trái sang phải?

Không thể tùy ý áp dụng Alpha-Beta thông thường trên cặp (u,v) vì nút Casper sử dụng hai quy tắc khác nhau. Tuy nhiên, nếu chỉ cần chọn hành động ở gốc và quy ước hòa ưu tiên nhánh trái: sau khi đã tính nhánh trái là (3,6), ở nhánh phải sau khi xét con đầu của Casper được (3,9), ta biết u của Casper bên phải không thể lớn hơn 3 (do u là min). Vì vậy gốc MAX không thể ưu tiên nhánh phải hơn nhánh trái theo u; có thể bỏ qua nhánh con (4,8) và các lá của nó. Cách cắt này giữ quyết định ở gốc với quy tắc phá hòa đã nêu, nhưng không cho giá trị (u,v) chính xác cho tất cả nút còn lại.

Nếu yêu cầu tính chính xác cặp (u,v) ở mọi nút hoặc không quy định cách phá hòa tại gốc, cần duyệt thêm và không thể khẳng định nhánh đó an toàn để bỏ qua cho mọi mục tiêu.

c) Nếu Pacman biết Casper thân thiện

Cả hai cùng tối đa hóa tiện ích, nên giá trị đạt được bằng giá trị lớn nhất trong các lá. Lá lớn nhất bằng 9, vì vậy trò chơi hợp tác có giá trị 9.

Bài 6. **Tính đầy đủ và tối ưu của các thuật toán tìm kiếm**

a) Breadth-First Search (BFS)

Đầy đủ nếu hệ số phân nhánh hữu hạn và tồn tại nghiệm ở độ sâu hữu hạn; trên đồ thị cần xử lý lặp trạng thái để tránh duyệt vòng. Tối ưu về số bước, do đó tối ưu về chi phí khi mọi bước có cùng một chi phí dương. Nếu trọng số cạnh khác nhau, BFS không nhất thiết cho đường ít chi phí nhất.

b) Depth-First Search (DFS)

Trên cây vô hạn hoặc có nhánh sâu vô hạn, DFS có thể không đầy đủ vì đi mãi trên một nhánh. Trên đồ thị hữu hạn với tập đã thăm đúng cách, DFS là đầy đủ đối với đích đạt được. DFS nhìn chung không tối ưu, kể cả khi mọi bước có chi phí bằng nhau.

c) Iterative Deepening Search (IDS)

Đầy đủ khi hệ số phân nhánh hữu hạn và nghiệm có độ sâu hữu hạn. IDS tìm thấy nghiệm ở độ sâu ít bước nhất, nên tối ưu khi chi phí các bước bằng nhau; nếu trọng số không đồng đều thì không bảo đảm tối ưu chi phí.

d) A* Search

Với $f(n) = g(n) + h(n)$, A* trên cây đầy đủ và tối ưu dưới các điều kiện thông thường: hệ số phân nhánh hữu hạn, chi phí mỗi bước được chặn dưới bởi $\varepsilon > 0$ và heuristic admissible, nghĩa là $h(n) \leq h^*(n)$ (thường lấy $h(goal) = 0$). Trên đồ thị, nếu không mở lại nút đã đóng thì cần

heuristic consistent; nếu cho phép cập nhật và mở lại nút khi tìm thấy đường tốt hơn, heuristic admissible có thể đủ để bảo đảm tối ưu cùng các giả định thích hợp.

e) Giải thích ảnh hưởng của chi phí và heuristic

Điều kiện bước có chi phí tối thiểu $\varepsilon > 0$ loại trừ việc mở rộng vô hạn các đường đi có tổng chi phí vẫn bị chặn. Heuristic admissible không đánh giá quá cao chi phí còn lại, bảo vệ việc tìm đúng nghiệm tối ưu. Heuristic consistent thỏa $h(u) \leq c(u,v) + h(v)$, khiến f không giảm dọc một đường đi; nhờ vậy A* graph search dễ xử lý trạng thái đã đóng mà không phải mở lại.

Bài 7. **Khi chi phí một cạnh thay đổi sau A***

Gọi C^* là chi phí tối ưu ban đầu, P^* là một đường tối ưu ban đầu. Giả sử trọng số của cạnh $X \to Y$ thay đổi từ c thành $c \pm n$, với $n > 0$ và trọng số sau cập nhật vẫn hợp lệ. Ta xét tác động lên đường tối ưu, đồng thời phân biệt trường hợp đã và chưa khám phá cạnh này.

a) Chi phí tăng n, cạnh nằm trên P^* và đã khám phá

Chi phí tối ưu mới có thể giữ nguyên hoặc tăng, nhưng không vượt $C^* + n$; tức $C^* \leq C' \leq C^* + n$. Tuy đường P^* trở nên đắt hơn n đơn vị, một đường thay thế có thể ít tốn kém hơn đường P^* mới. Vì vậy cần sửa các đường đi phụ thuộc vào cạnh $X \to Y$ và đối chiếu với các ứng viên khác, không thể chỉ cộng n vào mọi giá trị g đã lưu.

b) Chi phí giảm n, cạnh nằm trên P^* và đã khám phá

Đường P^* giảm đúng n đơn vị. Với đường đi đơn và không có chu trình âm, mọi đường còn lại đều không thể giảm nhiều hơn n so với chi phí cũ. Suy ra $C' = C^* - n$; P^* vẫn là một đường đi tối ưu và chỉ cần cập nhật lại chi phí của nó.

c) Chi phí tăng n, cạnh không nằm trên P^* nhưng đã khám phá

Chi phí tối ưu không đổi vì P^* vẫn có chi phí C^*; những đường đi dùng cạnh vừa tăng sẽ không trở nên tốt hơn. Có thể giữ nguyên đường cũ. Nếu còn lưu cấu trúc tìm kiếm, chỉ cần đánh dấu các giá trị phụ thuộc cạnh đã đổi là cần sửa khi tái sử dụng.

d) Chi phí giảm n, cạnh không nằm trên P^* nhưng đã khám phá

Chi phí mới có thể nhỏ hơn C^* nếu xuất hiện đường đi tốt hơn qua $X \to Y$; nếu không thì vẫn bằng C^*. Có thể dùng lại thông tin khoảng cách đã lưu, kiểm tra một đường qua $X \to Y$ với $g(X) + (c - n)$, rồi lan truyền các cập nhật tốt hơn từ Y tới những nút ảnh hưởng; cần cho phép mở lại nút nếu giá trị g được cải thiện.

e) Chi phí tăng n, cạnh không nằm trên P^* và chưa khám phá

Chi phí tối ưu không đổi: P^* vẫn có chi phí C^* và việc tăng một cạnh khác không thể tạo đường rẻ hơn. Không cần tìm lại đường đi chỉ để giữ nghiệm hiện tại.

f) Chi phí giảm n, cạnh không nằm trên P* và chưa khám phá

Chi phí tối ưu có thể giảm; vì cạnh chưa được tìm đến nên không thể kết luận ngay. Cần kiểm tra liệu có đường từ điểm đầu đến X rồi qua Y tới đích với tổng chi phí thấp hơn C*. Nếu có dữ liệu tìm kiếm trước đó, có thể bổ sung cạnh đã đổi và cập nhật tăng dần với LPA* hoặc D* Lite. Nếu dữ liệu không bao phủ vùng X, có thể phải tìm kiếm mở rộng hoặc chạy lại một phần đáng kể, nhưng không nhất thiết chạy toàn bộ từ đầu.

Tóm lại, tăng trọng số cạnh ngoài đường tối ưu cũ thì giữ nguyên kết quả; giảm trọng số cạnh trên đường tối ưu cũ thì giảm đúng n; các trường hợp còn lại cần xét các đường thay thế. Các thuật toán tìm kiếm tăng dần chỉ tiết kiệm công khi có lưu những thông tin cần thiết từ lần tìm trước.

Bài 8. **Tác tử thông minh và môi trường**

Đề xét ba lớp tác tử: A hành động theo f_A() và không dùng quan sát; B hành động theo f_B(percept) từ quan sát hiện tại; C lưu toàn bộ chuỗi percept và hành động theo f_C(percepts). Mục tiêu là xác định lớp nào đủ khả năng đưa ra hành động hợp lý trong những kiểu môi trường được nêu, với mô hình và thước đo thành tích đã xác định.

a) Quan sát đầy đủ, xác định, rời rạc, một tác tử, tĩnh

B và C có thể đủ năng lực. Khi percept hiện tại thể hiện đầy đủ trạng thái cần cho quyết định và các hành động là Markov theo trạng thái ấy, B có thể chọn hành động tối ưu từ quan sát hiện tại. C có thể làm tương tự bằng cách bỏ qua quá khứ. A không được bảo đảm vì không nhìn thấy trạng thái, nên không phân biệt được các tình huống yêu cầu hành động khác nhau.

b) Quan sát một phần, xác định, rời rạc, một tác tử, tĩnh

C là lớp cần thiết để bao quát những môi trường có hai tình huống cho cùng quan sát hiện tại nhưng đòi hỏi quyết định khác nhau theo lịch sử. Từ chuỗi percept, C có thể xác định trạng thái khả dĩ hoặc trạng thái niềm tin nếu mô hình cho phép. A và B đôi khi vẫn phù hợp với một môi trường cụ thể đơn giản, nhưng không bảo đảm trong mọi môi trường quan sát một phần.

c) Quan sát một phần, ngẫu nhiên, rời rạc, một tác tử, tĩnh

C vẫn là lớp có đủ thông tin quan sát để biểu diễn một chính sách phụ thuộc lịch sử; có thể dùng lịch sử để duy trì phân phối niềm tin về trạng thái và tối đa hóa lợi ích kỳ vọng. Tính ngẫu nhiên yêu cầu biết/xấp xỉ các xác suất chuyển và hàm thưởng. A và B không bảo đảm tối ưu cho toàn bộ lớp môi trường này.

d) Có tồn tại môi trường mà mọi tác tử đều hành động hợp lý không?

Đúng. Ví dụ môi trường chỉ có một hành động hợp lệ, hoặc môi trường mà mọi hành động ở mọi trạng thái đều mang lại cùng một phần thưởng và không ảnh hưởng đến kết quả tương lai. Lúc đó tác tử A, B, C hay bất kỳ tác tử nào cũng đạt giá trị tốt nhất có thể, nên tất cả đều hợp lý theo thước đo đã đặt.

--- Hết ---